DEEP REINFORCEMENT LEARNING · COMPANION READER

Introduction to Reinforcement Learning

Learning by trial, reward, and feedback: the agent, the environment, and the five elements that turn a goal into a policy.

Session 1 (Lecture 1) · Read alongside the lecture slides · Sutton & Barto, Ch. 1

How to use this companion. This reader expands each slide into plain English with one idea per section. The colour boxes guide you: **blue** intuition, **amber** everyday picture, **green** worked example, **red** watch out, **purple** key takeaway. Read it next to the slides, in order.

1 What this session covers

This first session introduces **Reinforcement Learning (RL)**: what it is, why it is different from other machine learning, and the vocabulary you will use for the whole course. We define RL as learning from reward and interaction, see when to use it, contrast it with supervised and unsupervised learning, and list its characteristics. We then meet the core cast — **agent**, **environment**, **state**, **action**, **reward** — and the four sub-elements: **policy**, **reward signal**, **value function**, and **model**. We close with the Tic-Tac-Toe example and your first learning rule, the **temporal-difference (TD)** update, plus the role of the step size α.

2 What is reinforcement learning?

Reinforcement learning is reward-based (or feedback-based) learning. An **agent** learns by interacting with an **environment**: it tries actions, sees what happens, and is guided by a numerical **reward**. It is *goal-oriented* learning based on interaction.

☞ The intuition

RL is not a type of neural network, nor an alternative to one — it is an *approach* for learning. The agent is not told the right answer. It discovers good behaviour by trying things and being rewarded or punished. Learning to win, not learning to copy.

★ Everyday picture

A child learns to ride a bicycle this way. Nobody hands them the equations of balance. They wobble (action), fall or stay up (reward), and adjust. Thousands of tiny trials, shaped by the reward of staying upright, produce a skilled rider.

A useful definition (Gartner): RL is based on rewarding desired behaviours or punishing undesired ones. Instead of one input producing one output, the algorithm produces a variety of outputs and is trained to select the right one based on certain variables.

2.1 When to use RL

RL fits large environments in three situations: (1) a *model* of the environment is known but an analytic solution is not available; (2) only a *simulation* model is given (simulation-based optimisation); or (3) the only way to gather information about the environment is to *interact* with it.

✗ Watch out

RL is powerful but expensive: it needs many interactions and careful reward design. If you have labelled examples of the right answer, supervised learning is usually simpler. Reach for RL when the task is sequential decision-making, not one-shot prediction.

✓ Key takeaway

RL is goal-oriented learning from interaction and reward — an *approach*, not a network — used when the agent must learn good behaviour by trying actions and seeing their consequences.

3 RL versus supervised and unsupervised learning

RL is the third great branch of machine learning, and it differs from the other two in what it learns from and what it learns to do.

Criterion	Supervised	Unsupervised	Reinforcement
Learns from	Labelled data	Unlabelled data	Interaction with environment
Data	Labelled	Unlabelled	No predefined data
Problems	Regression, classification	Clustering, association	Exploration / exploitation
Supervision	Full supervision	None	None (reward only)
Algorithms	Linear/logistic reg., SVM, KNN	K-means, Apriori	Q-learning, SARSA
Aim	Predict outcomes	Find patterns	Learn a series of actions

☞ The intuition

Supervised learning has a teacher who gives the correct answer for each example. Unsupervised learning has no teacher and only looks for structure. RL has no teacher either — only a *critic* who scores the outcome with a reward, often much later. The agent must figure out which actions earned the score.

★ Everyday picture

Three ways to learn cooking. Supervised: a chef stands beside you correcting every move. Unsupervised: you sort a pile of ingredients into groups with no goal. RL: you cook, taste the result (reward), and adjust next time — no one tells you the recipe.

✓ Key takeaway

Supervised learning predicts from labels, unsupervised finds patterns without labels, and RL learns a *sequence of actions* from reward alone — no labelled right answers, just consequences.

4 Characteristics of RL

RL problems share four defining features that make them harder than one-shot prediction.

- **No supervision** — only a real-valued reward signal, never the correct action.
- **Sequential decisions** — choices come in a sequence; each one shapes what comes next.
- **Time matters** — an action's value may depend on when it is taken.
- **Delayed feedback** — the reward for an action may arrive much later, not immediately.

★ Everyday picture

A chess move shows all four. No one tells you the best move (no supervision); each move sets up the next (sequential); timing matters; and you only learn the move was bad twenty moves later, when you lose (delayed feedback). Linking the late loss to the early mistake is the hard part — the **credit-assignment problem**.

✗ Watch out

Delayed reward is the central difficulty of RL. A reward at the end of a game must be shared back over all the moves that led to it. Most of the algorithms in this course exist to solve this credit-assignment problem.

✓ Key takeaway

RL is defined by reward-only feedback, sequential and time-dependent decisions, and delayed rewards — which together create the credit-assignment problem that the course's algorithms address.

5 The elements of an RL system

Beyond the **agent** and the **environment**, an RL system has four main sub-elements: a **policy**, a **reward signal**, a **value function**, and (optionally) a **model** of the environment.

5.1 Agent, environment, and the interaction loop

The **agent** is the learner and decision-maker. The **environment** is everything outside it. They interact in discrete time steps: the agent observes a **state**, takes an **action**, and the environment responds with a new state and a numerical **reward**. The goal is to maximise the **cumulative reward**.

- **Action** (A) — what the agent does at each step; A is the set of all possible moves (e.g. run left/right, jump, crouch).
- **State** (S) — the agent's current situation; after an action it moves to a new state.

- **Reward (R)** — the feedback for an action; positive for good outcomes, negative for bad.
- **Environment** — the outside world in which the agent operates.

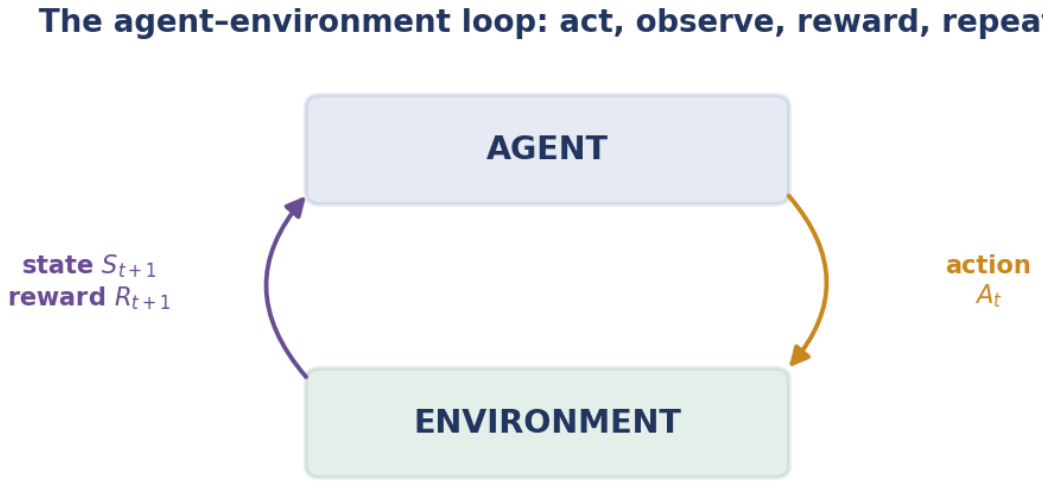

Figure 1: The agent–environment loop. At step t the agent sees state S_t and picks action A_t; the environment returns a new state S_{t+1} and reward R_{t+1}. The cycle repeats, and the agent learns to maximise total reward.

5.2 The notation

We write a set of states S, a set of actions A, and a set of rewards R. At each time step $t = 0, 1, 2, \ldots$, the agent receives a representation of the state $S_t \in S$ and selects an action $A_t \in A$, giving the **state–action pair** (S_t, A_t). The environment then transitions, and at step $t+1$ the agent receives the new state $S_{t+1} \in S$ and a reward $R_{t+1} \in R$ for the action A_t taken from state S_t. The formal goal of RL: take actions in states so as to maximise the notion of **cumulative reward**.

✗ Watch out

Note the index: the reward for action A_t is R_{t+1}, not R_t. The reward arrives *with* the next state, one step later. This off-by-one indexing trips up nearly everyone at first — watch it carefully in every formula.

5.3 Policy, value function, and model

Policy (π). The **policy** decides which action the agent takes in each state. It maps a state to probabilities over actions. If the agent follows policy π, then $\pi(a \mid s)$ is the probability of taking action a in state s:

$$\pi(a \mid s) = \Pr\left(A_t = a \mid S_t = s\right). \tag{1}$$

Value function. A **value function** measures how good it is to be in a state, or to take an action in a state — in terms of expected future reward. There are two kinds: the **state-value function** (how good a state is) and the **action-value function** (how good an action is in a state). These are the subject of later sessions.

Model. A **model** mimics the environment: given a state and action, it predicts the next state and reward. Models are used for **planning**. RL methods that use a model are **model-based**; those that learn purely from experience are **model-free**.

Element	Role	Human analogy
Policy	state → action (what to do)	a habit
Reward signal	the immediate, primary goal	pleasure / pain
Value function	expected long-term return from a state	judgement
Model	predicts next state & reward (optional)	imagination

☞ The intuition

The policy is the agent's *habit* (what to do). The value function is its *judgement* (how good a situation is). The model is its *imagination* (what might happen next). Learning is improving the habit using the judgement, optionally helped by the imagination.

✓ Key takeaway

An RL system has an agent and environment plus four sub-elements: the policy (what to do), the reward signal (the immediate goal), the value function (long-term goodness), and an optional model (for planning).

6 Tic-Tac-Toe and your first learning rule

The slides make RL concrete with Tic-Tac-Toe against an imperfect opponent. We want a player that finds the opponent's weaknesses and learns to maximise its chance of winning.

☞ The intuition

Keep a table of states (board positions), each with a **value** — our estimate of the probability of winning from that position. States that are wins get value 1; losses get 0; everything else starts at 0.5. We play many games and slowly update these values from experience.

We mostly move **greedily** to the state with the highest value (exploitation), but occasionally we move randomly to try something new (exploration). After a greedy move from state S_t to state S_{t+1}, we nudge the earlier state's value toward the later one. This is the **temporal-difference (TD) learning rule**:

$$V(S_t) \;\leftarrow\; V(S_t) \;+\; \alpha\,\big[\,V(S_{t+1}) - V(S_t)\,\big], \tag{2}$$

where $V(S_t)$ is the value of the state before the move, $V(S_{t+1})$ the value after, and α the **step size** (how big a nudge to take). The bracket $[V(S_{t+1}) - V(S_t)]$ is the **TD error** — the surprise between what we expected and what the next state says.

✎ Worked example: one TD update

Before a greedy move, $V(S_t) = 0.5$. The move leads to a strong position with $V(S_{t+1}) = 1.0$. Use step size $\alpha = 0.1$.

Step 1. TD error. $V(S_{t+1}) - V(S_t) = 1.0 - 0.5 = \mathbf{0.5}$ — a pleasant surprise; this move led somewhere better than expected.

Step 2. Scaled nudge. $\alpha \times 0.5 = 0.1 \times 0.5 = \mathbf{0.05}$.

Step 3. Update. $V(S_t) \leftarrow 0.5 + 0.05 = \mathbf{0.55}$.

Step 4. So what. The earlier state is now valued slightly higher, because it led to a better one. Repeat over many games and the values converge to true win probabilities.

6.1 What the step size α controls

The slides pose three questions about α over many games:

- $\alpha \to 0$ **gradually.** The updates shrink to nothing, so the values **converge** and stop changing — ideal against a *fixed* opponent.
- α **reduced but never** 0**.** The agent keeps learning a little forever, so it can **track** an opponent that slowly changes.
- α **kept constant.** The agent always weights recent games, so it *never fully converges* but adapts fastest to change.

✗ Watch out

A constant α never lets the estimates settle — good for a changing (non-stationary) world, bad if you want stable final values. The choice of α trades *stability* against *adaptability*. This exact trade-off returns in every later algorithm.

ML/AI connection. This TD update is the seed of the whole course. Q-learning, SARSA, and deep RL all update an estimate toward a slightly-better target using a step size — the same new $\leftarrow$ old $+\,\alpha[\text{target} - \text{old}]$ shape you just saw. Key takeaways from the example: we learn while interacting with the environment (the opponent), we have a clear goal, our policy maximises the chance of reaching it, and we exploit known values most of the time while exploring the rest.

✓ Key takeaway

The TD rule $V(S_t) \leftarrow V(S_t) + \alpha[V(S_{t+1}) - V(S_t)]$ nudges each state's value toward the next state's — learning from experience, one small step at a time, with α trading stability for adaptability.

7 Session recap

We defined RL as goal-oriented learning from interaction and reward, and saw when to use it. We placed it beside supervised and unsupervised learning, and named its four characteristics — reward-only, sequential, time-dependent, and delayed — which create the credit-assignment problem. We met the cast: agent and environment, plus policy, reward, value function, and model, and fixed the $(S_t, A_t, R_{t+1}, S_{t+1})$ notation. Finally, Tic-Tac-Toe gave us the temporal-difference update and the role of the step size α. Next session: the multi-armed bandit, the simplest setting where exploration and exploitation collide.